

Towards a Scalable Spiking Neural Network

Rasha Karakchi and Jacob Frierson

University of South Carolina, Columbia SC 29208, USA

karakchi@cec.sc.edu, jacobtf@email.sc.edu

Abstract. Spiking Neural Network (SNN) is the biological-brain model that has the capability of solving complex problems related to pattern recognition and character classification. Each neuron fires an output based on the discrete spikes that are collected from its predecessors which results in changing the ionic level or the membrane potential in the neuron. Once the level reaches a threshold, the neuron transmits signal to its successors. Transmitting data among neurons is a data-dependency problem because each neuron depends on receiving signals from its surrounding neurons at the current time and transmitting it into next neurons. Due to this high level of dependency, scaling up spiking neural network (represented by increasing number of neurons and hidden layers) is a real challenge. In hardware, the number of neurons is limited by the hardware (device) capacity and the number of internal wires available to connect between neurons. In this paper, we examine the main factors that significantly impact the scalability of spiking neural networks in hardware through implementing a spiking neural network model using the hardware description language SystemVerilog. Through evaluating the design on Alveo U55 high performance compute FPGA card, we found that the highest number of neurons that we can map on the hardware is 128 neurons per hidden layer and the higher number of synapses is 384,000 per hidden layer. We discussed the possible methods that can be exploited in order to improve scalability.

Keywords: Spiking Neural Network, Leaky Integrate-and-Fire, scalability, FPGA, accelerators.

I. Introduction

Spiking Neural Network (SNN) is the third generation of timing-based Artificial Neural Networks (ANNs) [1] where information is stored in the timing of spikes and its behavior is similar to the biological neurons in human brain. Like Conventional Neural Networks (CNNs), spiking neural network consists of a set of neurons that run in parallel. Each network consists of an input layer, hidden layers, and an output layer where each layer includes a set of neurons. More neurons in the network demands a higher level of parallelism and more power consumed [2,3], therefore, traditional computers represented by Von-Neumann architectures such as CPUs and GPUs struggle to execute such applications efficiently for large datasets. For this reason, recent research has shifted towards domain-specific architectures or accelerators that can be designed specifically for special purposes [2,3]. Domain-specific architectures are mainly classified as Application Specific Integrated Circuits (ASICs) and Field Programmable Gate Arrays (FPGAs). ASIC achieves higher performance than FPGA but at the cost of higher energy consumption and inflexibility [4,5]. FPGAs are two-dimensional arrays which consist of a massive number of processor elements (processors) where all can run in parallel and can be customized to perform different functions. This allowed FPGAs to be well-suited for neural network applications where each processor element can be programmed as a neuron. Achieving higher accuracy in neural networks is related to the number of neurons and the hidden layers the network consists of. The more neurons are utilized for training, the higher accuracy is achieved, therefore, scaling up the network is crucial to achieve better performance. However, there is a real challenge for neural network scalability in the hardware. One of the major factors which significantly impacts scaling up spiking neural network in FPGAs is the wire limitation, which is neglected in most of prior work [6-15]. As shown in Fig.1a, neurons are connected to their surroundings through wires. This number of wires is limited by the number of internal interconnections that are available in the device. Furthermore, the number of neurons is limited by the number of processor elements which, in turn, is limited by the available hardware resources. In this paper, we measure the limitation in number of wires (or synapses) represented by registers and number of processor elements (or neurons) represented by the configuration blocks through implementing a fully connected spiking neural network and evaluated it on Alveo U55 high performance compute FPGA card. We study the trade-off between the number of neurons in each hidden layer and the number of hidden layers which, in turn, affects the number of synapses in the network.

II. Proposed Design and Implementation

A. Proposed Spiking Neural Network (SNN) model.

We designed our SNN model based on Leaky Integrate-and-Fire (LIF) approach where neurons transmit information only when a membrane potential reaches the threshold as shown in Fig.1a. When the membrane potential reaches the threshold, a neuron generates and fires a signal into its successors, which in turn affects their potential based on this received signal. To model the membrane potential of biological neurons, each neuron in a SNN maintains and updates internal state with a neuron update function. The function combines the weighted pulses arriving on the synapses with the current state and produces a series of spikes to send downstream [10]. Spiking neural networks communicate with discrete values representing the spikes and transmitted among neurons through synapses shown in Fig.1a which shows an SNN network with the neuron's schematic.

B. Implementation of SNN Model on FPGA

FPGA is a two-dimensional array of logical elements called configurable logic blocks (CLBs) which communicate with each other through internal interconnections as shown in Fig.1b. The term “field-programmable” indicates that the FPGA’s abilities are adjustable and not hardwired by the manufacturer like other ICs [13]. We programmed the FPGA to serve as a spiking neural network model by mapping every neuron to a set of CLBs that represent a processor element using the hardware description language SystemVerilog. Fig.1b shows the hardware unit which consists of two adders that represent summation of spiking neuron and the membrane potential, and they are implemented using a register to store its dynamically changed value. Checking if the membrane potential exceeds the threshold is executed using an if-statement.

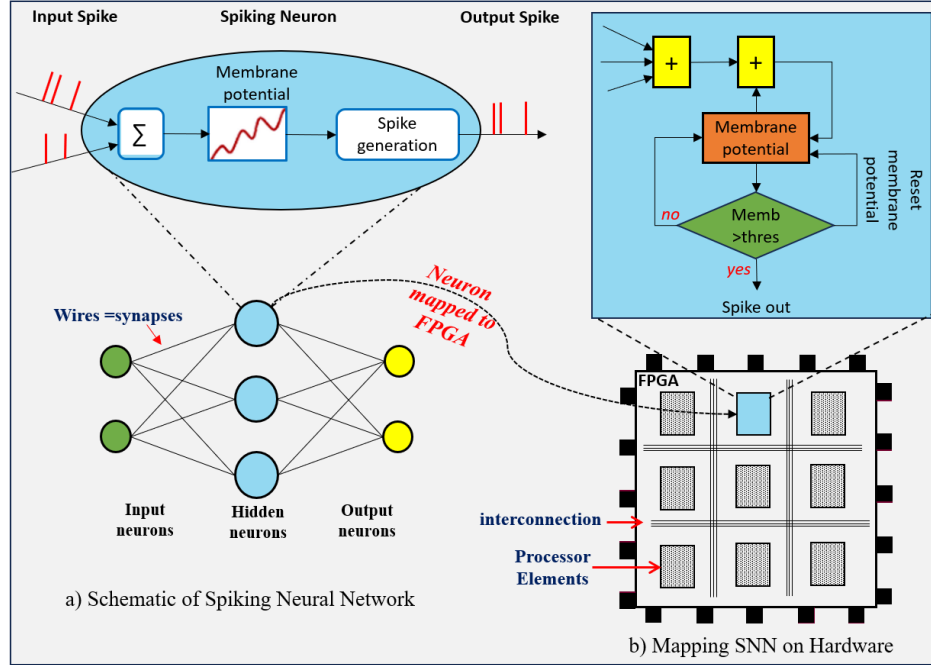


Fig. 1 Proposed SNN model and its Implementation. (a) SNN model and (b) mapping SNN on FPGA

III. Evaluation and Conclusion

We evaluated our SNN model on Alveo U55 high performance computer FPGA device, which consists of 1,304K CLBs and 2,607K registers. We designed our model as a parameterized model where we can vary the number of neurons and hidden layers. We set the weight width to 32-bit and membrane potential width to 48-bit. We set the number of output neurons to 10 while we set the inputs and the neurons in the hidden layer as parameters. Fig.2 shows the hardware resources consumed to implement 9 different models based on the 9 input values starting from the trivial 2 inputs up to 3000 inputs. For each input, we varied the number of hidden neurons between the trivial one neuron up to 128. Based on these two variables (inputs and hidden neurons), we measured the CLBs (Fig.2a) and registers (Fig. 2b). As shown in Fig.2 a and b, if we increase the number of hidden neurons to 500 inputs for 128-input model (last blue bar), the design will not fit on the device because it con-

sumes more than 100% of the device capacity (same for the registers in Fig.2b). While, for smaller models such as 16-hidden inputs model, more than 3000 inputs can fit on the device.

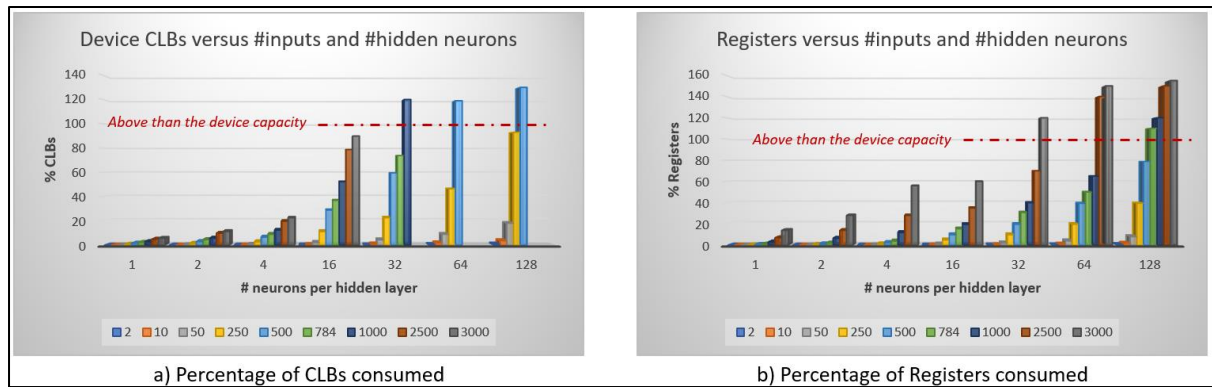


Fig. 2 Percentage of CLBs and registers consumed vs number of inputs and hidden neurons measured in CLBs (a) and registers (b).

For example, training an MNST image [16] which consists of 784 inputs requires using 32-neurons model which used 74% of the device CLBs and 31% of the registers, however at the cost of low accuracy. Using the fully-interconnection topology is a major issue in SNN scalability because it requires direct connections between neurons (unshared wires). For future work, we propose exploring hierarchical interconnections where multiple neurons can share the same path. This would reduce the hardware resources utilized and allow for more neurons to fit in the hardware.

References

1. Ankur Gupta and Lyle N.: Long. Hebbian learning with winner take all for spiking neural networks. In Proc. International Joint Conference on Neural Networks, 2009.[18]
2. C. Mead. 1990. Neuromorphic electronic systems. Proceedings of the IEEE 78, 10 (1990), 1629–1636
3. Bouvier, Maxence, et al. "Spiking neural networks hardware implementations and challenges: A survey." ACM Journal on Emerging Technologies in Computing Systems (JETC) 15.2 (2019): 1-35.
4. Hu, Yunxiang, Yuhao Liu, and Zhuovuan Liu. "A survey on convolutional neural network accelerators: GPU, FPGA and ASIC." 2022 14th International Conference on Computer Research and Development (ICCRD). IEEE, 2022.
5. Boutros, Andrew, Sadegh Yazdanshenas, and Vaughn Betz. "You cannot improve what you do not measure: FPGA vs. ASIC efficiency gaps for convolutional neural network inference." ACM Transactions on Reconfigurable Technology and Systems (TRETS) 11.3 (2018): 1-23.
6. Li, Jindong, et al. "FireFly: A high-throughput and reconfigurable hardware accelerator for spiking neural networks." arXiv preprint arXiv:2301.01905 (2023).
7. Han, Jianhui, et al. "Hardware implementation of spiking neural networks on FPGA." Tsinghua Science and Technology 25.4 (2020): 479-486.
8. Jin, Y., Liu, Y., and Li, P. 2016. SSO-LSM: A Sparse and Self-Organizing architecture for Liquid State Machine based neural processors. 55–60.
9. Schrauwen, B., D'Haene, M., Verstraeten, D., and Campenhout, J. V. 2008. Compact hardware liquid state machines on FPGA for real-time speech recognition. Neural Networks 21, 2 (2008), 511–523.
10. Wang, Q., Jin, Y., and Li, P. 2015. General-purpose LSM learning processor architecture and theoretically guided design space exploration. 1–4.
11. Maass, Wolfgang. "Networks of spiking neurons: the third generation of neural network models." Neural networks 10.9 (1997): 1659-1671.
12. https://en.wikipedia.org/wiki/Spiking_neural_network
13. <https://www.xilinx.com/products/silicon-devices/resources/programming-an-fpga-an-introduction-to-how-it-works.html>
14. David B. Thomas and Wayne Luk. Fpga accelerated simulation of biologically plausible spiking neural networks. In 17th IEEE Symposium on Field Programmable Custom Computing Machines, 2009.
15. Lei Wan, Yuling Luo, Shuxiang Song, J. Harkin, and Junxiu Liu. Efficient neuron architecture for fpga-based spiking neural networks. In 2016 27th Irish Signals and Systems Conference (ISSC), pages 1–6, June 2016.
16. <https://conx.readthedocs.io/en/latest/MNIST.html>